

Foundations of Bernoulli Types: A Unified Framework for Probabilistic Data Structures

Alexander Towell
atowell@siue.edu

July 29, 2025

Abstract

We present a unified mathematical framework for understanding probabilistic data structures through the fundamental distinction between *latent* (true but unobservable) values and their *observed* (noisy but measurable) approximations. This framework, based on Bernoulli types, provides a principled approach to reasoning about uncertainty in computation. We show that the relationship between latent and observed values can be characterized by confusion matrices, channel models, and Bayesian inference. Our key insight is that approximation is not merely an implementation detail but a first-class concept that can be lifted to the type level. We demonstrate that Bloom filters, sketches, randomized algorithms, and other probabilistic data structures are special cases of this general framework, unified by the latent-observed duality.

1 Introduction

1.1 The Fundamental Problem: Observing the Unobservable

In many computational contexts, we cannot directly access the values we care about. Instead, we work with noisy observations:

- A Bloom filter stores a *latent* set S , but we can only *observe* membership queries that may have false positives
- A sketch maintains a *latent* frequency distribution, but we can only *observe* approximate counts
- A randomized algorithm computes a *latent* result, but we can only *observe* a probabilistic output
- A compressed data structure represents *latent* information, but we can only *observe* lossy reconstructions

This paper formalizes the relationship between latent values and their observations through *Bernoulli types*—a type system where approximation is a first-class concept.

Consider the ubiquitous Bloom filter, which provides approximate set membership queries with one-sided error. While traditionally viewed as a specialized data structure, we show that it is merely one instance of a much broader class of Bernoulli types. By lifting approximation to the type level, we can reason about error propagation through complex systems in a principled manner.

1.2 Motivating Examples

Before formalizing the framework, we present three motivating examples that showcase different aspects of Bernoulli types:

Example 1.1 (Approximate Boolean). *The simplest non-trivial Bernoulli type is the approximate Boolean. Given a latent Boolean value $x \in \{\text{true}, \text{false}\}$, an observation $\tilde{x} \in \mathcal{B}(\mathbb{B}, 1)$ may differ from x with probability p . This models a binary symmetric channel where the observed value is a noisy version of the latent truth.*

Example 1.2 (Bloom Filter as Bernoulli Set). *A Bloom filter stores a latent set $S \subseteq U$ but only allows observations through membership queries. For any element $u \in U$, the observation " $u \in S$ " has:*

- If $u \in S$ (latent): always observes "yes" (no false negatives)
- If $u \notin S$ (latent): may observe "yes" with probability α (false positives)

Example 1.3 (Miller-Rabin as Bernoulli Map). *The Miller-Rabin primality test observes the latent property "is n prime?" through randomized testing. Given latent truth $\text{isPrime}(n)$, the observed result:*

- If n is prime (latent): always observes "prime"
- If n is composite (latent): may observe "prime" with probability $\leq 1/4^k$

2 Mathematical Foundations

2.1 Basic Definitions

Definition 2.1 (Latent and Observed Values). *For any type T :*

- A latent value $x \in T$ represents the true, unobservable value
- An observed value $\tilde{x} \in \mathcal{B}(T, n)$ represents a noisy observation of some latent value

We write $\tilde{x} \sim \mathcal{B}(T, n)(x)$ to denote that $\tilde{x}$ is an observation of latent value x .

Definition 2.2 (Bernoulli Type Constructor). *Given a type T , the Bernoulli type constructor $\mathcal{B}(T, n)$ produces a type whose values are observations of latent values in T , where observations are related to their latent values through an n -th order error model.*

The order n specifies the complexity of the error model:

- $n = 0$: Perfect observation (no errors, $\tilde{x} = x$ always)
- $n = 1$: Symmetric errors (observation errors independent of latent value)
- $n = 2$: Asymmetric errors (observation errors depend on latent value class)
- $n > 2$: Element-specific errors (observation errors depend on specific latent value)

Definition 2.3 (Observation Distribution). *For a Bernoulli type $\mathcal{B}(T, n)$ with error profile σ , the observation distribution specifies:*

$$\mathbb{P}[\tilde{x} = y | x] = P_\sigma(y | x) \quad (1)$$

where x is the latent value and y is the observed value.

2.2 Channel Model Interpretation

Bernoulli types can be understood through the lens of information theory as noisy channels:

Theorem 2.4 (Channel Correspondence). *Every Bernoulli type $\mathcal{B}\langle T, n \rangle$ with error profile σ corresponds to a discrete memoryless channel with input alphabet T , output alphabet T , and transition probabilities determined by σ .*

Proof. We establish a bijection between Bernoulli types and discrete memoryless channels (DMCs).

Forward direction: Given $\mathcal{B}\langle T, n \rangle$ with error profile σ , construct a DMC as follows:

- Input alphabet: $\mathcal{X} = T$
- Output alphabet: $\mathcal{Y} = T$
- Channel matrix: $W(y|x) = P_\sigma(y|x)$ where P_σ is determined by the error profile

For order $n = 1$ (symmetric), we have:

$$W(y|x) = \begin{cases} 1 - \epsilon & \text{if } y = x \\ \frac{\epsilon}{|T|-1} & \text{if } y \neq x \end{cases} \quad (2)$$

For order $n = 2$ (asymmetric), partition $T = T_+ \cup T_-$ and:

$$W(y|x) = \begin{cases} 1 - \alpha & \text{if } x \in T_-, y = x \\ \alpha & \text{if } x \in T_-, y \in T_+ \\ 1 - \beta & \text{if } x \in T_+, y = x \\ \beta & \text{if } x \in T_+, y \in T_- \end{cases} \quad (3)$$

Reverse direction: Given a DMC with $W(y|x)$, construct $\mathcal{B}\langle T, n \rangle$ by:

1. Determine the order n from the rank of $W - I$ where I is the identity matrix
2. Extract error parameters from the eigenvalues of W
3. Define the error profile σ from the channel transition probabilities

Memoryless property: The independence axiom of Bernoulli types ensures:

$$P(y_1, \dots, y_k | x_1, \dots, x_k) = \prod_{i=1}^k P(y_i | x_i) \quad (4)$$

This completes the correspondence. □

2.3 Confusion Matrix Representation

The channel correspondence naturally leads to a confusion matrix representation for Bernoulli types. For a type T with $|T| = n$ elements, the confusion matrix $Q \in \mathbb{R}^{n \times n}$ captures the complete error behavior:

Definition 2.5 (Confusion Matrix). *For a Bernoulli type $\mathcal{B}\langle T, k \rangle$, the confusion matrix $Q = [q_{ij}]$ is defined by:*

$$q_{ij} = \mathbb{P}[\text{observe } t_j | \text{latent } t_i] \quad (5)$$

where $t_1, \dots, t_n$ enumerate the elements of T .

Example 2.6 (Boolean Confusion Matrix). For $\mathcal{B}(\mathbb{B}, 2)$ with false positive rate α and false negative rate β :

latent \ observed	true	false
true	$1 - \beta$	β
false	α	$1 - \alpha$

This directly corresponds to a binary asymmetric channel (BAC).

Proposition 2.7 (Confusion Matrix Properties). For any Bernoulli type confusion matrix Q :

1. **Stochasticity:** Each row sums to 1: $\sum_j q_{ij} = 1$
2. **Order determination:** The order k satisfies $k \leq n(n-1)$
3. **Symmetry detection:** Order 1 iff $q_{ij} = \epsilon/(n-1)$ for all $i \neq j$
4. **Accuracy measure:** Overall accuracy = $\text{tr}(Q)/n$

2.4 Bayesian Inference: From Observed to Latent

The fundamental computational problem with Bernoulli types is inferring properties of latent values from observations:

Theorem 2.8 (Bayesian Inference for Bernoulli Types). Given an observation $\tilde{x} = y$ from $\mathcal{B}(T, n)$, the posterior probability of the latent value is:

$$\mathbb{P}[x = t_i | \tilde{x} = t_j] = \frac{q_{ij} \cdot \mathbb{P}[x = t_i]}{\sum_{k=1}^{|T|} q_{kj} \cdot \mathbb{P}[x = t_k]} \quad (6)$$

where q_{ij} is the confusion matrix entry and $\mathbb{P}[x = t_i]$ is the prior probability.

Example 2.9 (Boolean Inference). For $\mathcal{B}(\mathbb{B}, 2)$ with uniform prior $\mathbb{P}[x = \text{true}] = 0.5$:

$$\mathbb{P}[x = \text{true} | \tilde{x} = \text{true}] = \frac{1 - \beta}{1 - \beta + \alpha} \quad (7)$$

$$\mathbb{P}[x = \text{false} | \tilde{x} = \text{false}] = \frac{1 - \alpha}{1 - \alpha + \beta} \quad (8)$$

This shows how observation quality depends on both error rates.

Corollary 2.10 (Multiple Independent Observations). Given n independent observations $\tilde{x}_1, \dots, \tilde{x}_n$ of the same latent value:

$$\mathbb{P}[x = t | \tilde{x}_1, \dots, \tilde{x}_n] \propto \mathbb{P}[x = t] \prod_{i=1}^n \mathbb{P}[\tilde{x}_i | x = t] \quad (9)$$

As $n \rightarrow \infty$, the posterior concentrates on the true latent value (assuming non-degenerate confusion matrix).

3 The Bernoulli Type Hierarchy

3.1 Order 0: Deterministic Types

When $n = 0$, we have $\mathcal{B}(T, 0) \cong T$. This represents the degenerate case where no approximation occurs, corresponding to a perfect channel with zero error rate.

3.2 Order 1: Symmetric Error Models

First-order Bernoulli types model situations where observation errors occur with equal probability regardless of the latent value:

$$\mathbb{P}[\tilde{x} \neq x] = \epsilon \quad \forall x \in T \quad (10)$$

where $\tilde{x} \sim \mathcal{B}\langle T, 1 \rangle(x)$ is an observation of latent value x . This corresponds to a symmetric noisy channel where all values are equally difficult to observe accurately.

3.3 Order 2: Asymmetric Error Models

Second-order types distinguish between different kinds of observation errors based on the latent value. For Boolean types:

$$\mathbb{P}[\tilde{x} = \text{false} | x = \text{true}] = \beta \quad (\text{false negative rate}) \quad (11)$$

$$\mathbb{P}[\tilde{x} = \text{true} | x = \text{false}] = \alpha \quad (\text{false positive rate}) \quad (12)$$

This asymmetry is crucial for modeling real-world systems:

- Bloom filters: $\beta = 0$ (never miss latent members)
- Medical tests: often $\alpha \ll \beta$ (minimize false alarms)
- Security systems: often $\beta \ll \alpha$ (minimize missed threats)

3.4 Higher Orders: Element-Specific Errors

For orders $n > 2$, error rates can depend on the specific element being approximated. This models situations where certain values are more difficult to represent accurately than others.

4 Primitive Bernoulli Types

4.1 Void and Unit Types

The most fundamental types have trivial Bernoulli approximations:

Proposition 4.1 (Degenerate Bernoulli Types). *For the void type $\perp$ and unit type $\top$:*

$$\mathcal{B}\langle \perp, n \rangle \cong \perp \quad \forall n \quad (13)$$

$$\mathcal{B}\langle \top, n \rangle \cong \top \quad \forall n \quad (14)$$

Proof. The void type has no values to approximate, and the unit type has only one value with no uncertainty. $\square$

4.2 Sum Types

Sum types (disjoint unions) have multiple approximation strategies:

Definition 4.2 (Bernoulli Sum Types). *Given types A and B , we distinguish:*

1. **First-order:** $\mathcal{B}\langle(A + B), n\rangle$ - uniform error across variants
2. **Higher-order:** $\mathcal{B}\langle A, n\rangle + \mathcal{B}\langle B, m\rangle$ - independent errors per variant
3. **Mixed:** $\mathcal{B}\langle A, n\rangle + B$ - partial approximation

Example 4.3 (Option Types). *For $\text{Option}[T] = T + \top$:*

- $\mathcal{B}\langle \text{Option}[T], 1\rangle$: May confuse **Some**(x) with **None**
- $\text{Option}[\mathcal{B}\langle T, n\rangle]$: Only the contained value is approximate

4.3 Product Types

Product types compose approximations component-wise:

Theorem 4.4 (Product Error Propagation). *For latent value $(a, b) \in A \times B$ with independent observations $\tilde{a} \sim \mathcal{B}\langle A, n\rangle(a)$ and $\tilde{b} \sim \mathcal{B}\langle B, m\rangle(b)$:*

$$\mathbb{P}[(\tilde{a}, \tilde{b}) = (a', b')] = \mathbb{P}[\tilde{a} = a' | a] \cdot \mathbb{P}[\tilde{b} = b' | b] \quad (15)$$

4.4 Function Types

Functions are the most complex primitive type to approximate:

Definition 4.5 (Bernoulli Function Types). *A Bernoulli approximation of $f : A \rightarrow B$ can be:*

1. $\mathcal{B}\langle f, n\rangle : A \rightarrow B$ - probabilistic implementation
2. $f : \mathcal{B}\langle A, n\rangle \rightarrow \mathcal{B}\langle B, m\rangle$ - lifted to Bernoulli types
3. $\mathcal{B}\langle (A \rightarrow B), n\rangle$ - function chosen from distribution

5 Algebraic Properties

5.1 Functorial Structure

Theorem 5.1 (Bernoulli is a Functor). *The Bernoulli type constructor $\mathcal{B}^n$ forms an endofunctor on the category **Type** of types and functions, preserving composition and identities up to the specified error bounds.*

Proof. To prove $\mathcal{B}^n$ is a functor, we must show:

1. **Type mapping:** For each type T , we have $\mathcal{B}^n(T) = \mathcal{B}\langle T, n\rangle$
2. **Function mapping:** For each function $f : A \rightarrow B$, we have $\mathcal{B}^n(f) : \mathcal{B}^n(A) \rightarrow \mathcal{B}^n(B)$
3. **Identity preservation:** $\mathcal{B}^n(\text{id}_T) = \text{id}_{\mathcal{B}^n(T)}$ up to error ϵ

4. **Composition preservation:** $\mathcal{B}^n(g \circ f) = \mathcal{B}^n(g) \circ \mathcal{B}^n(f)$ up to composed error

Function mapping: Given $f : A \rightarrow B$, define:

$$\mathcal{B}^n(f)(a) = \text{sample from } P(f(a') | a' \sim a) \quad (16)$$

Identity: For identity function $\text{id}_T : T \rightarrow T$:

$$\mathcal{B}^n(\text{id}_T)(t) = t \text{ with probability } 1 - \epsilon \quad (17)$$

Composition: For $f : A \rightarrow B$ and $g : B \rightarrow C$:

$$\mathcal{B}^n(g \circ f)(a) = \mathcal{B}^n(g)(\mathcal{B}^n(f)(a)) \quad (18)$$

$$= \text{sample from composed distribution} \quad (19)$$

The error accumulates according to the composition law proven below. $\square$

5.2 Composition Laws

When composing Bernoulli types, errors propagate in predictable ways:

Proposition 5.2 (Error Propagation for Composition). *For functions $f : A \rightarrow B$ and $g : B \rightarrow C$ with symmetric error rates ϵ_f and ϵ_g , the composition has error rate:*

$$\epsilon_{g \circ f} = \epsilon_f + \epsilon_g - \epsilon_f \epsilon_g \quad (20)$$

Proof. The probability of correct computation through the composition is:

$$P(\text{correct}) = P(f \text{ correct}) \cdot P(g \text{ correct} | f \text{ correct}) \quad (21)$$

$$= (1 - \epsilon_f)(1 - \epsilon_g) \quad (22)$$

$$= 1 - \epsilon_f - \epsilon_g + \epsilon_f \epsilon_g \quad (23)$$

Therefore, the error rate is:

$$\epsilon_{g \circ f} = 1 - P(\text{correct}) = \epsilon_f + \epsilon_g - \epsilon_f \epsilon_g \quad (24)$$

Note this is the same formula as for the union of independent events, reflecting that an error occurs if either f or g produces an error. $\square$

Corollary 5.3 (Iterated Composition). *For n functions with identical error rate ϵ :*

$$\epsilon_{\text{composed}} = 1 - (1 - \epsilon)^n \approx n\epsilon \text{ for small } \epsilon \quad (25)$$

5.3 Monadic Structure

Bernoulli types form a monad, capturing the computational pattern of observing latent values:

Theorem 5.4 (Bernoulli Monad). *The triple $(\mathcal{B}^n, \eta, \mu)$ forms a monad where:*

- $\eta : T \rightarrow \mathcal{B}^n(T)$ embeds a latent value as a perfect observation
- $\mu : \mathcal{B}^n(\mathcal{B}^n(T)) \rightarrow \mathcal{B}^n(T)$ flattens nested observations

Intuitively:

- $\eta(x)$ creates an observation that perfectly reflects latent value x
- μ handles "observing an observation"—multiple layers of noise

Proof. We verify the monad laws:

1. Left identity: $\mu \circ \eta_{\mathcal{B}^n(T)} = \text{id}_{\mathcal{B}^n(T)}$

Given $x \in \mathcal{B}^n(T)$:

$$(\mu \circ \eta)(x) = \mu(\eta(x)) \quad (26)$$

$$= \mu(\text{pure } x) \quad (27)$$

$$= x \quad (28)$$

2. Right identity: $\mu \circ \mathcal{B}^n(\eta) = \text{id}_{\mathcal{B}^n(T)}$

3. Associativity: $\mu \circ \mu_{\mathcal{B}^n(T)} = \mu \circ \mathcal{B}^n(\mu)$

The key insight is that flattening nested Bernoulli types combines their error rates multiplicatively. $\square$

Definition 5.5 (Kleisli Composition). *For Bernoulli computations $f : A \rightarrow \mathcal{B}^n(B)$ and $g : B \rightarrow \mathcal{B}^m(C)$:*

$$(g \circ_K f)(a) = \mu((\mathcal{B}^m(g))(f(a))) \quad (29)$$

5.4 Algebraic Laws

Theorem 5.6 (Bernoulli Algebra). *Bernoulli types satisfy modified algebraic laws:*

1. **Approximate Associativity:** $(a \star b) \star c \approx_\epsilon a \star (b \star c)$
2. **Approximate Commutativity:** $a \star b \approx_\epsilon b \star a$ (when applicable)
3. **Approximate Identity:** $a \star e \approx_\epsilon a$ where e is the identity

where $\approx_\epsilon$ denotes equality up to error rate ϵ .

6 Implementation Considerations

6.1 Type Erasure

In practice, we often need to work with collections of Bernoulli types with different underlying implementations. Type erasure enables this flexibility:

```
template <typename T>
class bernoulli_value {
    struct concept {
        virtual T approximate() const = 0;
        virtual double error_rate() const = 0;
    };
    // ... implementation details
};
```


6.2 Expression Templates

Complex operations on Bernoulli types can be optimized using expression templates, delaying computation until the final result is needed:

```
auto expr = bernoulli_set<int> | union_op | intersection_op;  
// Computation happens only when expr is evaluated
```

7 Related Work

The concept of approximate computation has been explored from various angles:

- **Probabilistic Data Structures:** Bloom filters [1], Count-Min sketch [2], and HyperLogLog [4] are specific instances of Bernoulli types.
- **Approximate Computing:** Work on approximate hardware [5] and programming languages [6] shares similar goals but lacks our unified type-theoretic framework.
- **Differential Privacy:** The Laplace mechanism can be viewed as implementing Bernoulli types over real numbers with specific error distributions [3].

8 Conclusions and Future Work

We have presented Bernoulli types as a unified framework based on the fundamental distinction between latent (true) values and their observations. This perspective transforms approximation from a necessary evil into a first-class concept. Our contributions include:

- **Conceptual Framework:** The latent/observed duality provides intuitive understanding of all probabilistic data structures
- **Mathematical Foundation:** Confusion matrices and Bayesian inference connect observations to latent values
- **Compositional Theory:** Error propagation through complex systems follows predictable laws
- **Unification:** Bloom filters, sketches, and randomized algorithms are revealed as instances of a general pattern

The key insight is that in many computational contexts, we cannot directly access the values we care about—we can only observe them through a noisy channel. By making this explicit at the type level, we gain powerful tools for reasoning about approximate computation.

Future work includes:

- Extending the framework to continuous types and quantum computation
- Developing type inference algorithms for optimal error profiles
- Implementing a full programming language with native Bernoulli type support
- Exploring hardware acceleration for Bernoulli type operations

References

- [1] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [2] Graham Cormode and S Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. In *Journal of Algorithms*, volume 55, pages 58–75. Elsevier, 2005.
- [3] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*, pages 265–284. Springer, 2006.
- [4] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. In *Discrete Mathematics and Theoretical Computer Science*, pages 137–156, 2007.
- [5] Jie Han and Michael Orshansky. Approximate computing: An emerging paradigm for energy-efficient design. In *2013 18th IEEE European Test Symposium (ETS)*, pages 1–6. IEEE, 2013.
- [6] Adrian Sampson, Werner Dietl, Emily Fortuna, Danushen Gnanapragasam, Luis Ceze, and Dan Grossman. Enerj: Approximate data types for safe and general low-power computation. In *ACM SIGPLAN Notices*, volume 46, pages 164–174. ACM, 2011.